

Chapter Four

Pointers

Pointer

- A pointer is a variable that holds the *address* of something else (variable or a value).

- **Syntax** :

 dataType *identifier;

 e.g.

int *x;

 x is a pointer to an integer.

 float *Z; z is a pointer to float

 double *y; y is a pointer to double.

- Addresses: garbage, something, nothing

- When created:

int * q;

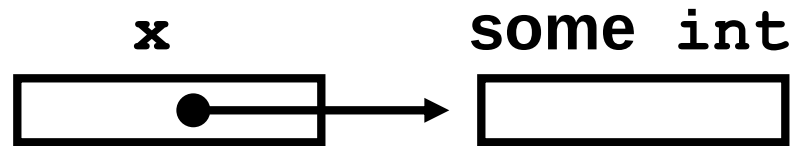
q = 0; // now it points to nothing

q = NULL; // not portable, use 0 instead

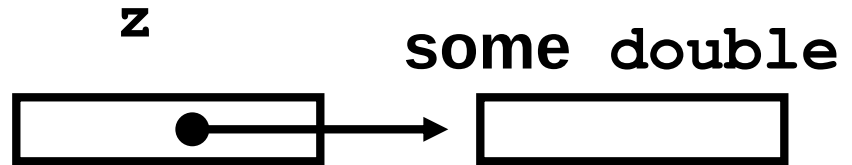
q = nullptr; // C++11 and above

Pointers to anything

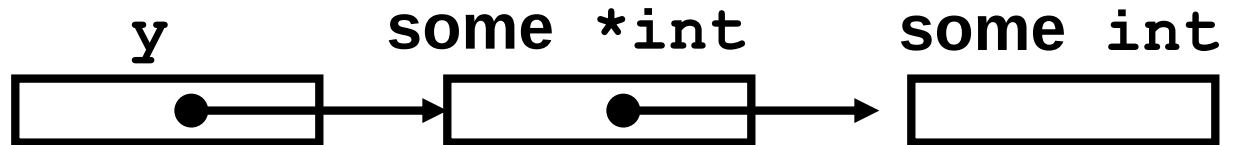
```
int *x;
```



```
double *z;
```



```
int **y;
```



Address operator(&)

- To assign a value to the pointer variable
 - use the address operator &

• E.g.

```
int foo;
```

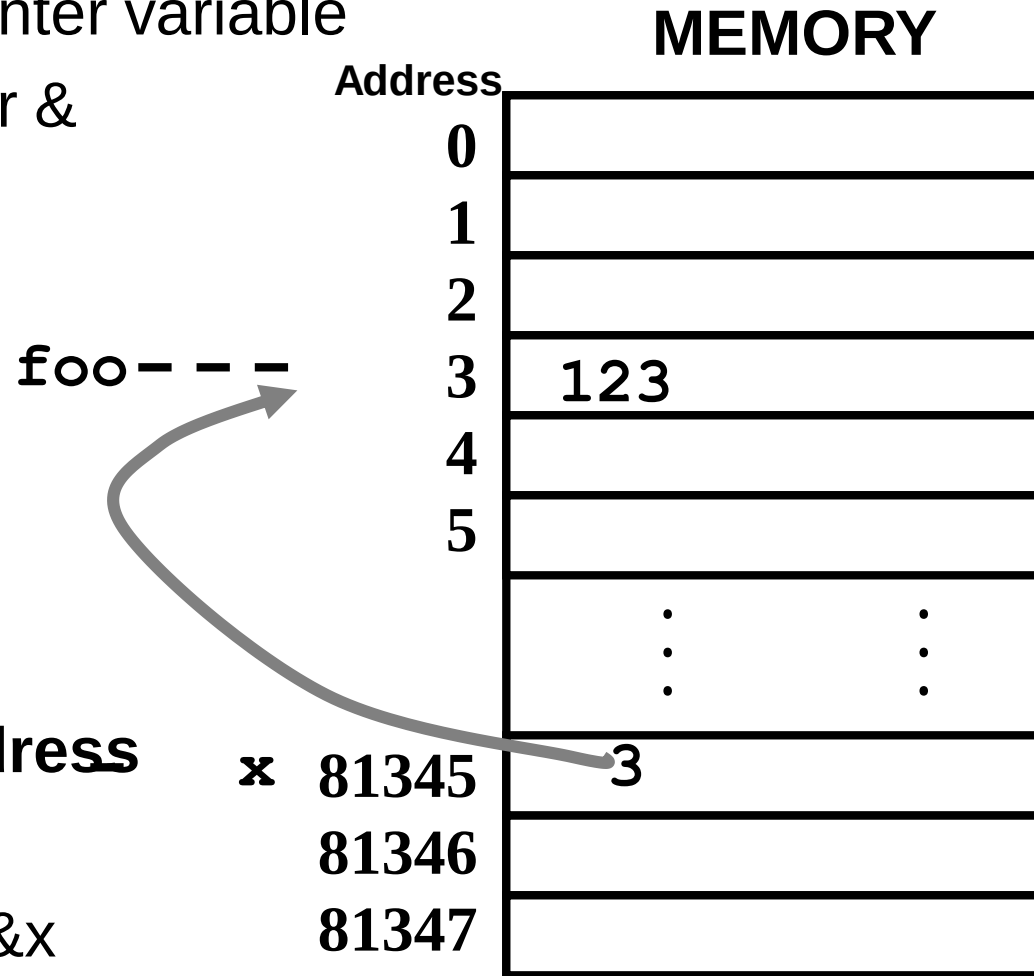
```
int *x;
```

```
foo = 123;
```

```
x = &foo;
```

- To access memory address of a variable:

e.g `cout<< &foo; cout<<&x`



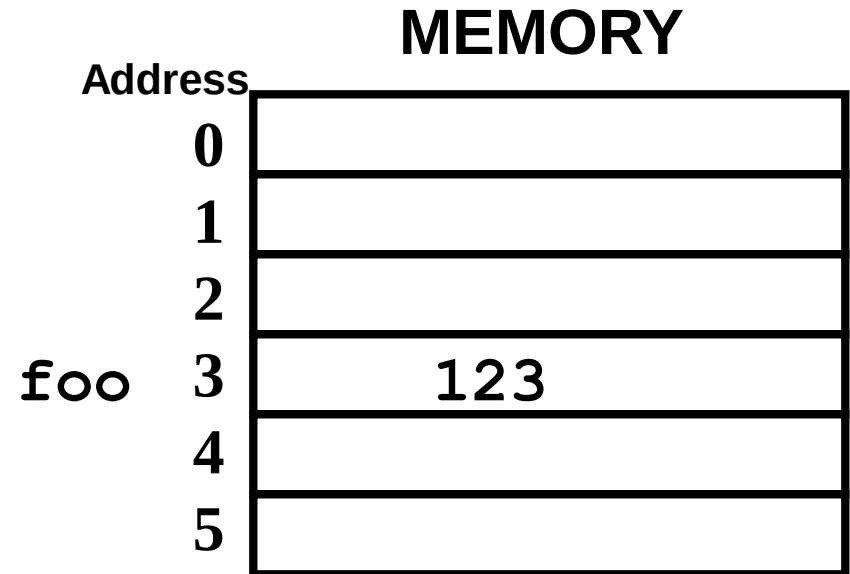
&foo

In C++ you can get the *address* of a variable with the “&” operator.

```
int foo;
```

```
foo = 123;
```

```
x = &foo;
```



- **Int &ref=y** means “ref is alliance of y”
- Reference must always refer to something
 - Must be initialized, cannot be changed

Pointer to Anything

A **void pointer** (void*) is a special pointer that can store the address of **any type of variable**:

```
void *p; // p can point to anything
```

You can store the address of an int, float, char, struct, array, etc. on a void pointer.

- `int a = 10;`
- `float b = 3.5;`
- `char c = 'A';`
- `void *p;`
- `p = &a; // ok`
- `p = &b; // ok`
- `p = &c; // ok`

Dereferencing operator (*)

- * is used to get values stored at address and to assign a value to a pointer variable.
- To assign a value to the variable foo through the pointer variable

- we must use the dereferencing operator *

- E.g

```
*x=10;
```

```
//Changes the value of foo from 123 to 10
```

```
Equivalent to      foo=10;
```

- You can use the integer x points to in a C++ expression like this:

```
y = *x + 17;
```

- ```
*x = *x + 1;
```

To dereference a void pointer, you must cast.

```
int a = 10;
```

```
void *p = &a;
```

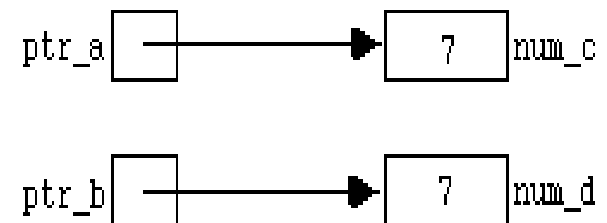
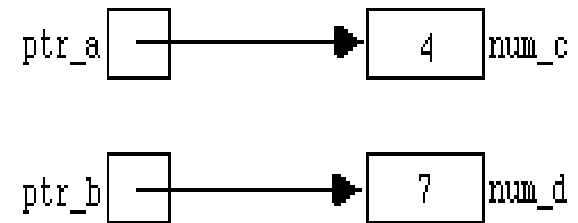
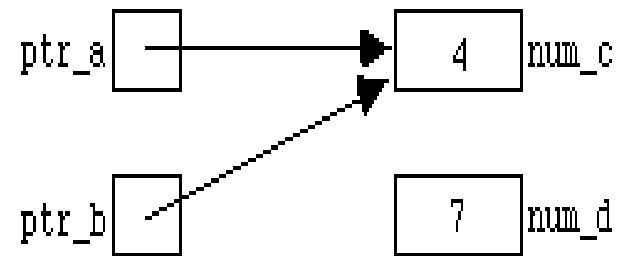
```
cout << *(int*)p; // casting
```

```
#include <iostream.h>
```

```
int main() {
 int *ptr_a, *ptr_b;
 int num_c = 4, num_d = 7;
 ptr_a = &num_c;
 ptr_b = ptr_a;
 cout << *ptr_a << " " << *ptr_b << "\n";
 ptr_b = &num_d;
 cout << *ptr_a << " " << *ptr_b << "\n";
 *ptr_a = *ptr_b;
 cout << *ptr_a << " " << *ptr_b << "\n";
 cout << num_c << " " <<
 &&num_c << "\n";
 return 0; }
```

- The output of this program is:

- 4 4
- 4 7
- 7 7
- 7 7





# More Pointers

- *int i = 50;*

*int j = 75;*

*int \*p1 ; int \* p2*

*p1 = &i ; p2 = &j;*

*cout << \*p1;*

*p1 = p2 ; \*p2 =0;*

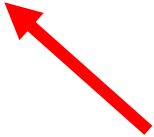
*cout <<\*p1;*

# Assigning a value to a *dereferenced* pointer

A pointer must have a value before you can *dereference* it (follow the pointer).

```
int *x;
*x=3;
```

**ERROR!!!**  
x doesn't point to anything!!!



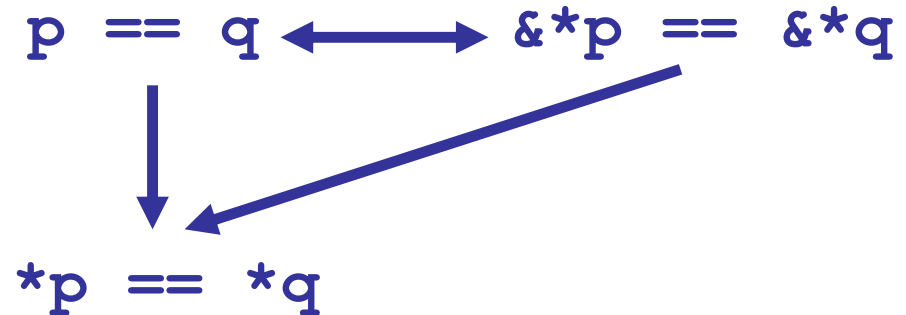
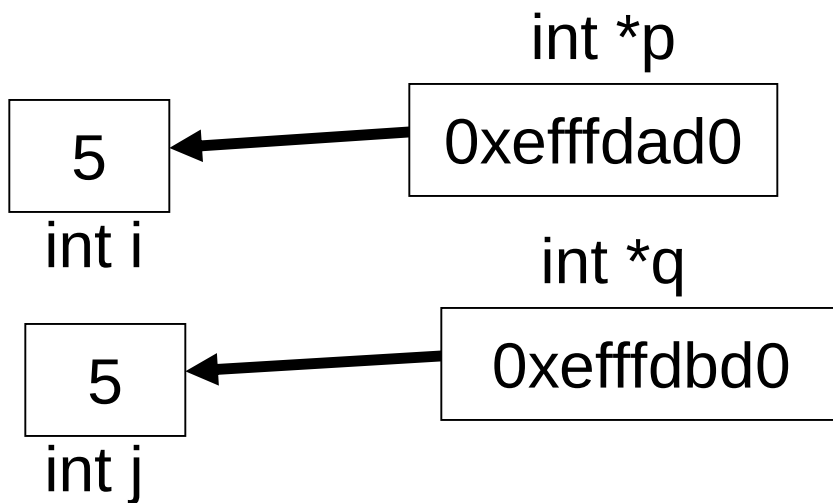
```
int foo;
int *x;
x = &foo;
*x=3;
```

this is fine  
x points to foo



# Location and Value Comparisons

- Pointers may be compared for equality
  - Same as comparing addresses of what pointers point to (memory locations:)
  - Contents of what pointers point to may be compared, too
- First implies second, not other way around



# Const Pointers and Pointers to Const Types

```
int main (){
 const int i = 0;
 int j = 1;
 int k = 2;

 // pointer to int
 int * w = &j;

 // const pointer to int
 int * const x = &k;

 // pointer to const int
 const int * y = &i;

 // const pointer to const int
 const int * const z = &j;
}
```

- Make promises via the **const** keyword in pointer declaration:
  - not to change pointer itself
  - not to change value it aliases
  - can also promise neither/both
- Read declarations right to left
- In this example, can change
  - **w** and what it points to
  - what **x** points to but not **x**
  - **y** but not what it points to
  - neither **z** nor what it points to
- A pointer to a non-const type cannot point to a const variable
  - **w** and **x** can't point to **i**

# Pointers and arrays

- The name of an array is a pointer to the 0th element of the array (the beginning of the array)

```
int array[5] ;
```

```
// array is equivalent to & array[0]
```

```
*array = 5; is like array[0] = 5;
```

```
int j = *(array+4) is like int j = array[4]
```

```
cout << *array; is like cout << array[0];
```

# Pointers and Arrays

- An array name is basically a *const* pointer.
- You can use the `[]` operator with a pointer:

```
int *x;
```

```
int a[10];
```

```
x = &a[2];
```

```
for (int i=0; i<3; i++)
```

```
 x[i]++;
```

**x** is “the address of **a[2]**”



**x[i]** is the same as **a[i+2]**



# Pointers

- Summary
  - `&p`, `p`, and `*p` all have different meanings
  - `&p` means the address of `p`
  - `p` means the content of `p`
  - `*p` means the content pointed to by `p`, that is pointed to by the content of memory location

# Pointer Arithmetic

- can add an integer quantity to or subtract an integer quantity from a pointer.
- the outcome depends on the size of the object pointed to
- *not* the same as integer arithmetic
  - $\text{ptr} + 1 \rightarrow$  moves the pointer by **sizeof(\*ptr) bytes**
  - $\text{ptr} - 1 \rightarrow$  moves backward by **sizeof(\*ptr) bytes**

`int x = 10;`

`int *p = &x;`

If: address of x = 1000

`sizeof(int) = 4`

•Then:

`p = 1000`

`p + 1 = 1004`

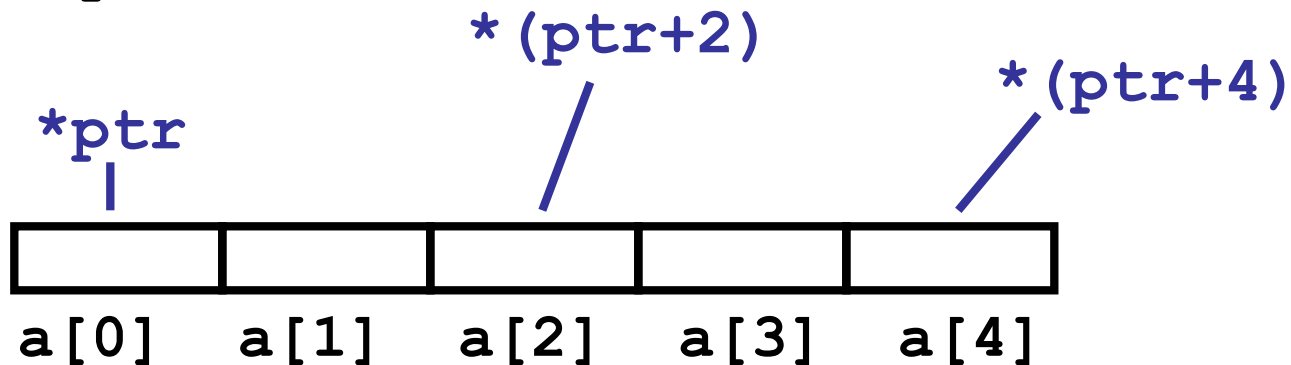
`p + 2 = 1008`



# Pointer arithmetic

- Integer math operations can be used with pointers.
- If you increment a pointer, it will be increased by the size of whatever it points to.

```
int *ptr = a;
```



```
int a[5];
```

# printing an array

```
void print_array(int a[], int len) {
 for (int i=0;i<len;i++)
 cout << "[" << i << "]" = "
 << a[i] << endl;
}
```

array version

```
void print_array(int *a, int len) {
 for (int i=0;i<len;i++)
 cout << "[" << i << "]" = "
 << *a++ << endl;
}
```

pointer version

- `a++` // moves to next element
- `*a++` // access value at `a[0]` and then move the pointer
- `(*a)++` // increases the value at `a[0]` but not pointer

- If `++` is closer to `p`, the pointer moves
- If `++` is closer to `*p`, the value changes

- `*p++` → use value, move pointer

- `(*p)++` → change value, pointer stays

- `*++p` → move pointer, then use value

`++*p` → `(++, *)` have the same precedence. Associativity is **right-to-left**. So `++*p = ++(*p)`

Precedence order (high → low):

1. `()`

2. postfix `++`

3. unary `*`

So:

- `*p++` → `*(p++)`

- `(*p)++` → value first, then increment

# Dynamic memory

- Dynamic memory is memory that is **allocated at runtime (during program execution)** from the **heap**, instead of at compile time from the stack.
- Memory for dynamic objects
  - Requested from the free store
    - Free store is memory controlled by operating system
  - ◆ Operation specifies
    - The type and number of objects
  - ◆ If there is sufficient memory to satisfy the request
    - A pointer to sufficient memory is returned by the operation
  - ◆ If there is insufficient memory to satisfy the request
    - An exception is generated
      - ◆ An *exception* is an error state/condition which if not handled (corrected) causes the program to terminate

# The Basic New Form

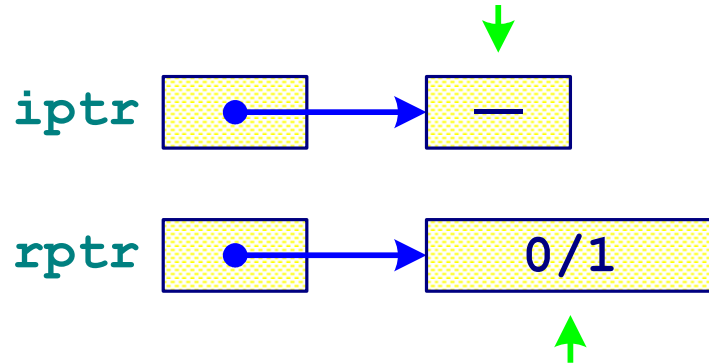
- A **dynamic object** is an **object created in dynamic memory (heap)** using **new** keyword.
- A dynamic object is an object whose memory is allocated dynamically.
- Syntax
  - Ptr = new SomeType ;**
  - Where
    - Ptr is a pointer of type SomeType
- Beware
  - The newly acquired memory is uninitialized unless there is a default SomeType constructor

# Examples

```
int *iptr = new int;
```

```
Rational *rptr = new Rational;
```

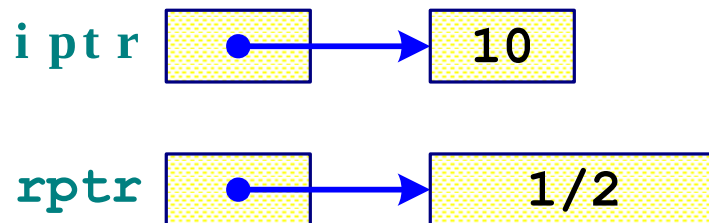
Uninitialized int object



Rational object with default initialization

```
int *iptr = new int(10);
```

```
Rational *rptr = new Rational(1,2);
```



# The Primary New Form

- Syntax

**P = new SomeType [Expression] ;**

- Where

- P is a pointer of type SomeType
- Expression is the number of contiguous objects of type SomeType to be constructed -- we are making a list

- Note

- The newly acquired list is initialized if there is a default SomeType constructor

- Because of flexible pointer syntax

- P can be considered to be an array

# Examples

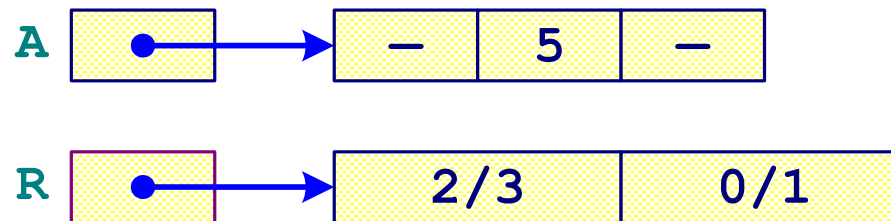
```
int *A = new int [3];
```

```
Rational *R = new Rational[2];
```

```
A[1] = 5;
```

```
Rational r(2/3);
```

```
R[0] = r;
```





# Delete Operators

- Delete is a keyword to delete/remove/free memory space.
- Forms of request

`delete P; // used if storage came from new`

`delete [] P; // used if storage came  
from new[]`

- Storage pointed to by P is returned to free store
  - P is now undefined

# Cleaning Up

```
int n;
cout << "Enter list size: ";
cin >> n;
int *A = new int[n];
GetList(A, n);
SelectionSort(A, n);
DisplayList(A, n);
delete [] A;
```

# Classwork

```
• int a[] = {5, 10, 15};
int *p = a;
int x = *p++;
int y = (*p)++;
cout<<x<<y<<a[1];
x = *++p;
cout<<"-----";
cout<<x;
cout<< (*p)++;
cout<<*p;;
*p++ = 100;
cout<<endl;
cout<<a[0]<<a[1]<<a[2];
int *ptr3=a;
for (int i=0;i<=2;i++)
 cout<<ptr3[i];
```

|        |     |   |
|--------|-----|---|
| 00A132 | 5   | q |
| 00A128 | 20  |   |
| 00A136 | 35  |   |
| 00A124 | 78  |   |
| 00A120 | 19  | p |
| 00A116 | 10  |   |
| 00A112 | 200 |   |
| 00A108 | 300 |   |
| 00A104 | 4   |   |
| 00A100 | 150 |   |

- Create a pointer variable m that point to the variable p;
- Change the value of p to 20 using the pointer variable p.
- Cout<< ++m<< m++;
- Cout<< ++\*m<< ++(\*m)<< ++\*(m);
- Cout<< m<< m+3<< \*(m-2);
- Write a statement that indicates the pointer m to points to q rather than p.

```

int var=10;
int *ptr=&var;
*ptr=20;
int *a=&var;
int **ptr2=&ptr;
**ptr2=30;
cout<<var;
cout<<++(*a);
cout<<**ptr2;
int v[3] = {10, 100, 200};
int *ptr;
 ptr = v;
 for (int i = 0; i < 3; i++){
cout << "Value at ptr = "<<ptr<<"\n";
 cout <<"Value at *ptr="<< *ptr << "\n";
 ptr++;
 }
cout<<ptr[0]<< ptr[1]<< ptr[2];

```